

Stress-Testing n-step Soft Actor-Critic under Limited Compute

Final RL Project Report, extended version

Radmir I.

2026

Contents

1	Abstract	2
2	Introduction	2
3	Research question and contribution	3
4	Related work	3
4.1	Soft Actor-Critic	3
4.2	N-step returns and credit assignment	3
4.3	Off-policy multi-step correction	3
4.4	SACn and T-SAC	4
5	Method	4
5.1	Standard SAC components	4
5.2	N-step transition buffer	4
5.3	What is fixed in the ablation	5
6	Experimental setup	5
6.1	Environments	5
6.2	Hyperparameters	5
6.3	Hardware	6
7	Results	6
7.1	Pendulum-v1 sanity check	6
7.2	HalfCheetah-v5 main ablation	7
7.3	Sample-efficiency markers	7
8	Discussion	8
9	Threats to validity	8
9.1	Statistical validity	8
9.2	Algorithmic validity	8
9.3	Hyperparameter fairness	8
9.4	Environment coverage	8
9.5	Compute budget	9
10	Reproducibility checklist	9

11	Future work	9
12	Conclusion	9
	References	10

Short version. This project tests one central mechanism behind recent n-step SAC work: whether replacing the standard 1-step critic target with an n-step target helps Soft Actor-Critic under a limited compute budget. On HalfCheetah-v5, the 3-step variant improved learning compared with the 1-step baseline, while the 5-step variant failed under the same hyperparameters. The main conclusion is deliberately narrow: the n-step horizon behaves like a stability parameter, not like a simple bigger-is-better knob.

1 Abstract

This project studies a compact n-step modification of Soft Actor-Critic (SAC) under a limited compute budget. The main question is whether replacing the standard 1-step critic target with n-step targets improves sample efficiency in continuous-control tasks, and where the horizon becomes unstable. I implement a configurable n-step buffer before the replay buffer and compare SAC with $n = 1$, $n = 3$, and $n = 5$. The experiments include Pendulum-v1 as a sanity check and HalfCheetah-v5 as the main benchmark. On HalfCheetah-v5, $n = 3$ reaches the best final return and reaches high-return behavior earlier than the baseline, while $n = 5$ fails to learn a strong running policy under the same hyperparameters. The result supports the idea that n-step credit assignment can help SAC, but only within a stability-sensitive range.

2 Introduction

Soft Actor-Critic is a strong off-policy actor-critic algorithm for continuous-control tasks. It combines replay-based off-policy updates with a stochastic actor and a maximum-entropy objective, encouraging the policy to maximize both return and entropy (Haarnoja, Zhou, Abbeel, et al. 2018; Haarnoja, Zhou, Hartikainen, et al. 2018). This makes SAC a natural baseline for continuous-control experiments: it is sample-efficient, relatively stable, and already hard to improve without careful changes.

In standard SAC, the critic is usually trained with a 1-step temporal-difference target. This target is simple and stable, but reward information is propagated backward only one step at a time. N-step returns propose a more aggressive alternative: accumulate several rewards before bootstrapping. This can improve credit assignment because useful reward information can move backward faster (Sutton and Barto 2018).

However, SAC is off-policy. Transitions sampled from the replay buffer may have been collected by older policies. A longer n-step target depends on a longer replay fragment and may therefore introduce more off-policy bias and target variance. This is the core reason why n-step SAC is non-trivial and why recent work has revisited the topic (Łyskawa, Lewandowski, and Wawrzyński 2025; Tian, Celik, and Neumann 2025).

The project is intentionally not a state-of-the-art claim. It is a bounded reproduction and ablation study: I isolate one mechanism - the critic target horizon - and test whether the expected trade-off appears in a compact, reproducible setup.

3 Research question and contribution

The main research question is:

Do n-step returns improve SAC under limited compute, and how sensitive is the result to the chosen horizon?

The project compares three variants:

Method	Description	Expected behavior
SAC $n = 1$	standard 1-step SAC baseline	stable baseline
SAC $n = 3$	moderate 3-step target	may improve sample efficiency
SAC $n = 5$	longer 5-step target	potentially higher risk

The contribution is small but concrete:

1. a configurable n-step transition buffer integrated into a compact SAC implementation;
2. a controlled $n = 1/3/5$ ablation on Pendulum-v1 and HalfCheetah-v5;
3. committed raw logs, plots, summary tables, and reproducibility notes;
4. a failure analysis of the $n = 5$ run under fixed hyperparameters.

4 Related work

4.1 Soft Actor-Critic

SAC is an off-policy maximum-entropy actor-critic algorithm. The actor is optimized not only for expected return but also for entropy, which encourages exploration and can improve robustness. The practical SAC formulation also uses twin Q-critics, target networks, replay buffer sampling, and automatic entropy temperature tuning (Haarnoja, Zhou, Abbeel, et al. 2018; Haarnoja, Zhou, Hartikainen, et al. 2018).

SAC is a good baseline for this project because it is already a strong continuous-control method. If a small n-step modification helps or breaks SAC, the effect is meaningful enough to analyze.

4.2 N-step returns and credit assignment

N-step returns are a classical temporal-difference learning idea. Instead of bootstrapping after a single reward, an n-step target accumulates several rewards before bootstrapping:

$$G_t^{(n)} = r_t + \gamma r_{t+1} + \dots + \gamma^{n-1} r_{t+n-1} + \gamma^n V(s_{t+n}).$$

This can propagate delayed rewards faster than a 1-step target. At the same time, the target contains more sampled rewards and changes the bias-variance trade-off (Sutton and Barto 2018; Sharma et al. 2017).

4.3 Off-policy multi-step correction

The off-policy setting is the main difficulty. In replay-based learning, longer trajectory fragments may have been collected by a behavior policy different from the current target policy. Several multi-step methods explicitly address this issue through correction or trace truncation. $\text{Retrace}(\lambda)$ is designed as a safe and efficient off-policy return-based method (Munos et al.

2016). IMPALA introduces V-trace to stabilize learning under decoupled acting and learning (Espeholt et al. 2018). These works are not implemented here, but they motivate why naive off-policy multi-step targets should be treated carefully.

A related warning comes from the “deadly triad” of function approximation, bootstrapping, and off-policy learning (Hasselt et al. 2018). Naive n-step SAC touches all three: it uses neural critics, bootstrapped targets, and replayed off-policy transitions.

4.4 SACn and T-SAC

SACn directly studies how to combine Soft Actor-Critic with n-step returns. Its motivation is close to this project: n-step returns can speed up learning, but the usual combination with SAC introduces bias due to changing action distributions, and importance sampling may become numerically unstable (Łyskawa, Lewandowski, and Wawrzyński 2025).

T-SAC / Chunking the Critic takes a more ambitious direction. It uses a sequence-conditioned critic with trajectory chunks and aggregated n-step targets, making temporal context part of critic learning rather than only changing a scalar target (Tian, Celik, and Neumann 2025).

This project does not reproduce SACn or T-SAC. Instead, it isolates a smaller shared mechanism: changing the critic target horizon. This makes the experiment feasible on limited compute and easier to interpret.

5 Method

5.1 Standard SAC components

The implementation keeps the usual SAC structure:

- squashed Gaussian actor;
- twin Q-critics;
- target critic networks;
- replay buffer;
- automatic entropy tuning;
- deterministic evaluation checkpoints.

The critic update uses the minimum of the two target critics and subtracts the entropy term:

$$V_{\text{target}}(s') = \min_i Q_i^{\text{target}}(s', a') - \alpha \log \pi(a'|s').$$

For the standard 1-step version, the target is:

$$y_t = r_t + \gamma(1 - d_t)V_{\text{target}}(s_{t+1}).$$

5.2 N-step transition buffer

The only algorithmic component varied in the main ablation is the transition aggregation before the replay buffer. The n-step buffer stores a short queue of recent transitions. When enough transitions are available, it builds an aggregated transition:

```
state      = s_t
action     = a_t
reward_n   = r_t + gamma r_{t+1} + ... + gamma^(k-1) r_{t+k-1}
next_state = s_{t+k}
```

`discount_n = gamma^k`
`done` = terminal flag inside the n-step window

Here $k \leq n$. If the episode ends before the full window is collected, the window is cut early. For $n = 1$, this construction reduces to standard SAC.

The critic target becomes:

$$y_t^{(n)} = R_t^{(n)} + (1 - d_t)\gamma^k V_{\text{target}}(s_{t+k}).$$

The replay buffer therefore samples the same kind of objects for all runs, but the reward and discount fields encode different target horizons.

5.3 What is fixed in the ablation

The comparison keeps the same architecture and hyperparameters for $n = 1$, $n = 3$, and $n = 5$. This choice is intentional. It makes the experiment a clean sensitivity check of the target horizon.

This also means that the comparison is not fully tuned for every horizon. A separate hyperparameter search might improve the $n = 5$ run. The current result should therefore be read as a fixed-configuration stability test, not as proof that $n = 5$ can never work.

6 Experimental setup

6.1 Environments

Two environments are used.

Environment	Role	Steps	Seed
Pendulum-v1	sanity check	30,000	0
HalfCheetah-v5	main benchmark	100,000	0

Pendulum-v1 is a simple classic-control continuous-action task with action space `Box(-2.0, 2.0, (1, ), float32)` and observation shape `(3, )` (Farama Foundation 2026b). HalfCheetah-v5 is a MuJoCo continuous-control locomotion task with action space `Box(-1.0, 1.0, (6, ), float32)` and observation shape `(17, )`; the goal is to make the 2D robot run forward as fast as possible (Farama Foundation 2026a).

6.2 Hyperparameters

Parameter	Value
Discount factor gamma	0.99
Target update coefficient tau	0.005
Initial entropy coefficient alpha	0.2
Automatic entropy tuning	yes
Actor learning rate	3e-4
Critic learning rate	3e-4
Alpha learning rate	3e-4

Parameter	Value
Batch size	256
Replay buffer size	1,000,000
Hidden sizes	[256, 256]
Start steps	5,000
Update after	5,000
Evaluation interval	5,000 steps
Evaluation episodes	5
Compared horizons	$n = 1, 3, 5$

All completed runs use deterministic evaluation with five episodes per checkpoint. Raw CSV logs and plots are committed to the repository.

6.3 Hardware

The experiments were run on a Ryzen 5700 laptop through WSL Ubuntu using CPU training. This is intentionally a limited-compute setup. The project is designed as a reproducible student experiment rather than a large-scale benchmark.

7 Results

7.1 Pendulum-v1 sanity check

On Pendulum-v1, all three variants train successfully and reach similar final quality. The main role of this experiment is to verify that the implementation works and that the n -step target construction does not immediately break SAC on a simple environment.

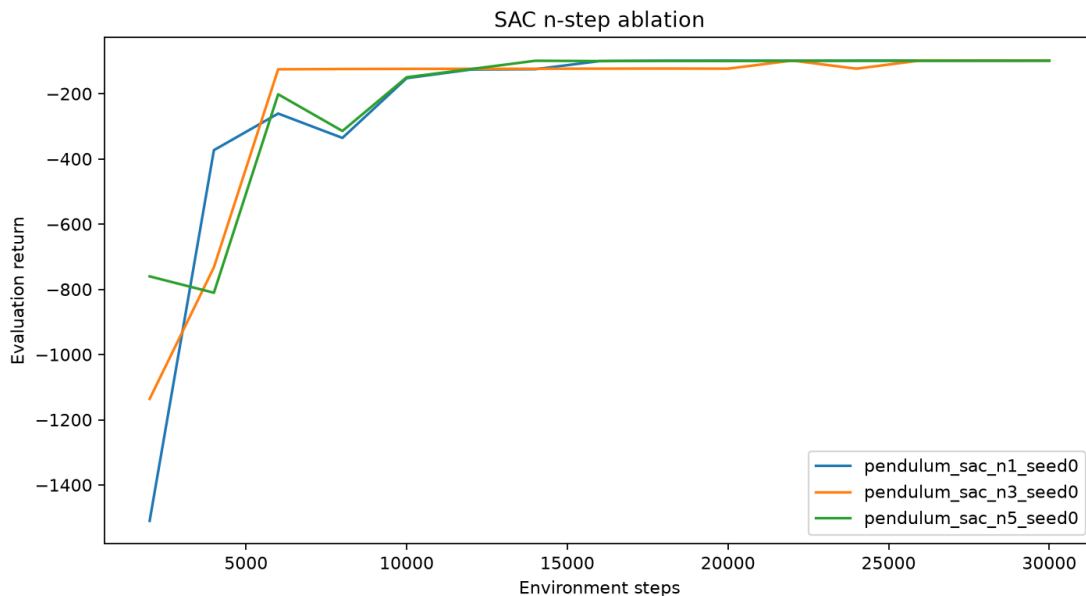


Figure 1: Pendulum-v1 learning curves

The curves suggest that $n = 3$ reaches a good return quickly, but Pendulum-v1 is too simple to support a strong claim about the best horizon. This experiment should be treated as a sanity

check, not as the main evidence.

7.2 HalfCheetah-v5 main ablation

The main result is observed on HalfCheetah-v5.

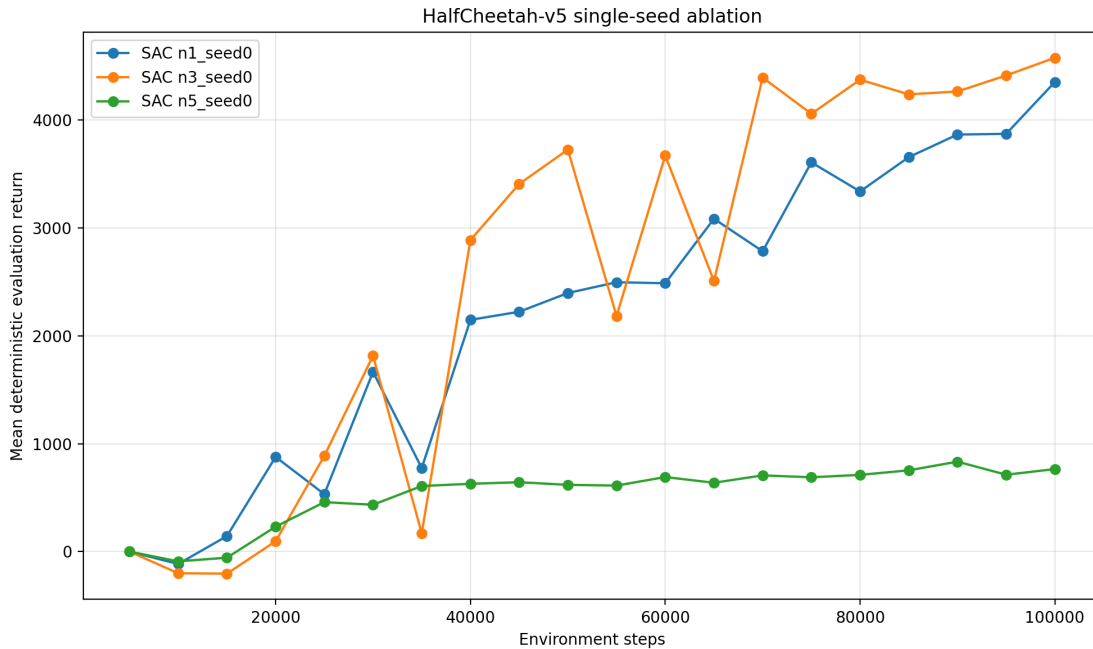


Figure 2: HalfCheetah-v5 learning curves

Method	Final eval return	Best eval return
SAC $n = 1$	4354.49	4354.49
SAC $n = 3$	4578.91	4578.91
SAC $n = 5$	764.49	833.37

Main observation. In this single-seed run, $n = 3$ gives the best final return and reaches strong performance earlier than the 1-step baseline. The $n = 5$ variant fails to learn a strong running policy under the same settings.

7.3 Sample-efficiency markers

Final return is not the only useful metric. The HalfCheetah learning curve also shows a difference in how quickly the variants reach strong behavior.

Method	Steps to return ≥ 3000	Steps to return ≥ 4000
SAC $n = 1$	65,000	100,000
SAC $n = 3$	45,000	70,000
SAC $n = 5$	not reached	not reached

These markers support the visual reading of the curve: $n = 3$ learns faster in this run, while $n = 5$ never reaches high-return behavior.

8 Discussion

The result supports the expected trade-off. A moderate n -step horizon can improve sample efficiency because reward information is propagated backward faster. This likely explains why $n = 3$ improves over the 1-step baseline on HalfCheetah-v5.

At the same time, a longer naive target can increase off-policy bias and target variance. This likely explains why $n = 5$ fails under the same hyperparameters. The most useful result is not only that $n = 3$ obtains the best final return. The more informative result is the failure of $n = 5$: it shows that increasing the target horizon can change the stability of off-policy SAC under fixed hyperparameters.

The safest conclusion is narrow: in this limited-compute single-seed ablation, 3-step SAC improved HalfCheetah learning compared with the 1-step baseline, while 5-step SAC failed under the same hyperparameters. The project does not claim universal superiority of $n = 3$.

This failure mode matches the motivation behind recent n -step SAC work. N -step targets can help credit assignment, but the horizon interacts with off-policy bias, target variance, critic stability, and entropy dynamics.

9 Threats to validity

9.1 Statistical validity

The main limitation is the use of a single seed. The current experiment is useful as bounded reproduction and ablation evidence, but it is not enough to make a statistical claim that $n = 3$ is generally better than $n = 1$, or that $n = 5$ generally fails. A stronger study should use at least 3-5 seeds and report confidence intervals.

9.2 Algorithmic validity

The implementation uses naive n -step targets without off-policy correction. This is intentional because the project tests the direct effect of changing the critic target horizon. However, it also means that the failure of $n = 5$ should not be interpreted as a failure of all n -step SAC methods. SACn-style correction or sequence-aware critics could stabilize longer horizons.

9.3 Hyperparameter fairness

All horizons use the same hyperparameters. This makes the ablation clean because only the target horizon changes. At the same time, it may disadvantage $n = 5$. A longer horizon may require different learning rates, target smoothing, entropy settings, or replay settings.

9.4 Environment coverage

The project uses Pendulum-v1 and HalfCheetah-v5. Pendulum-v1 is mainly a sanity check; it is too simple to support a strong claim about the best horizon. HalfCheetah-v5 is the main benchmark, but one MuJoCo task is not enough for a general conclusion. Future work should add Hopper-v5, Walker2d-v5, and possibly Ant-v5.

9.5 Compute budget

All experiments were run under laptop CPU compute. This shaped the design: short training budgets, single seed, and compact architecture. The project should be read as a small reproducible ablation study rather than a paper-scale benchmark.

10 Reproducibility checklist

Item	Status
Code committed	yes
Configs committed	yes
Raw CSV logs committed	yes
Plots committed	yes
Summary table committed	yes
Random seed recorded	yes, seed 0
Hardware documented	yes
Limitations documented	yes
Full multi-seed statistics	no

Key repository artifacts:

- implementation: `src/rl/nstep_buffer.py`, `src/rl/sac.py`, `src/train.py`;
- configs: `configs/pendulum_sac_n*.yaml`, `configs/halfcheetah_sac_n*.yaml`;
- raw logs: `results/raw/*.csv`;
- plots: `results/plots/*.png`;
- summary: `results/halfcheetah_summary.csv`;
- slides and notes: `slides/`, `docs/`.

11 Future work

The most useful extensions are:

1. run 3-5 seeds;
2. add more MuJoCo environments such as Hopper-v5, Walker2d-v5, and Ant-v5;
3. report confidence intervals and failure rates;
4. compare naive n-step SAC with SACn-style corrections;
5. tune hyperparameters separately for each horizon;
6. test sequence-aware critics inspired by T-SAC.

12 Conclusion

This project implements configurable n-step targets in SAC and tests them in a small but reproducible ablation. Pendulum-v1 confirms that the implementation works. HalfCheetah-v5 shows the main trade-off: $n = 3$ helps, $n = 1$ remains a strong baseline, and $n = 5$ fails under the same hyperparameters.

Final takeaway. The n-step horizon is a stability parameter, not a bigger-is-better knob.

References

- Espeholt, Lasse, Hubert Soyer, Rémi Munos, Karen Simonyan, Volodymyr Mnih, Tom Ward, Yotam Doron, et al. 2018. "IMPALA: Scalable Distributed Deep-RL with Importance Weighted Actor-Learner Architectures." *arXiv Preprint arXiv:1802.01561*. <https://arxiv.org/abs/1802.01561>.
- Farama Foundation. 2026a. "Gymnasium Documentation: HalfCheetah-V5." https://gymnasium.farama.org/environments/mujoco/half_cheetah/.
- . 2026b. "Gymnasium Documentation: Pendulum-V1." https://gymnasium.farama.org/environments/classic_control/pendulum/.
- Haarnoja, Tuomas, Aurick Zhou, Pieter Abbeel, and Sergey Levine. 2018. "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor." *arXiv Preprint arXiv:1801.01290*. <https://arxiv.org/abs/1801.01290>.
- Haarnoja, Tuomas, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, et al. 2018. "Soft Actor-Critic Algorithms and Applications." *arXiv Preprint arXiv:1812.05905*. <https://arxiv.org/abs/1812.05905>.
- Hasselt, Hado van, Yotam Doron, Florian Strub, Matteo Hessel, Nicolas Sonnerat, and Joseph Modayil. 2018. "Deep Reinforcement Learning and the Deadly Triad." *arXiv Preprint arXiv:1812.02648*. <https://arxiv.org/abs/1812.02648>.
- Łyskawa, Jakub, Jakub Lewandowski, and Paweł Wawrzyński. 2025. "SACn: Soft Actor-Critic with n-Step Returns." *arXiv Preprint arXiv:2512.13165*. <https://arxiv.org/abs/2512.13165>.
- Munos, Rémi, Tom Stepleton, Anna Harutyunyan, and Marc G. Bellemare. 2016. "Safe and Efficient Off-Policy Reinforcement Learning." *arXiv Preprint arXiv:1606.02647*. <https://arxiv.org/abs/1606.02647>.
- Sharma, Sahil, Girish Raguvir J, Srivatsan Ramesh, and Balaraman Ravindran. 2017. "Learning to Mix n-Step Returns: Generalizing Lambda>Returns for Deep Reinforcement Learning." *arXiv Preprint arXiv:1705.07445*. <https://arxiv.org/abs/1705.07445>.
- Sutton, Richard S., and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press.
- Tian, Dong, Onur Celik, and Gerhard Neumann. 2025. "Chunking the Critic: A Transformer-Based Soft Actor-Critic with n-Step Returns." *arXiv Preprint arXiv:2503.03660*. <https://arxiv.org/abs/2503.03660>.